Thread-Based PyTorch DataLoader on Free-Threaded Python

Team: James Tian (jamestia), Yueqi Ren (yueqir)

URL: https://yijiangt.github.io/projects/15418-dataloader/

Summary

We are going to build a thread-based backend for PyTorch's `DataLoader` on free-threaded Python 3.14 with work-stealing scheduling, backpressure, and zero-copy collation, and compare it against multiprocess loading on GHC and NCSA Delta in throughput, CPU efficiency, memory, and tail latency.

Background

PyTorch's `torch.utils.data.DataLoader` parallelizes data loading only with worker processes. The main process assigns whole batches of sample indices to workers round-robin through per-worker index queues. Each worker calls `dataset[i]` for every index (file read, decode, augmentation), collates the samples into a batch in newly allocated shared memory, and sends it back through a multiprocessing queue. The main process reorders batches so they are delivered in sampler order, optionally copying them into pinned memory, and caps in-flight work at `prefetch_factor x num_workers` batches.

Per-sample work is independent, so the workload is data-parallel. For image pipelines (JPEG decode plus augmentation), it is CPU-bound and benefits directly from more cores. Processes were the only option because the GIL serializes Python threads. Python 3.14 makes the free-threaded (no-GIL) build officially supported, and PyTorch, torchvision, and Pillow now ship wheels for it, so a thread-based loader that shares one address space is now possible.

The process design has known costs that threads can remove:

- **Memory replication:** reading Python objects updates their reference counts, so copy-on-write duplicates dataset metadata in every worker.
- **IPC and serialization:** non-tensor fields are pickled, and every batch crosses a process boundary.
- **Startup latency:** workers are forked or spawned, so the first batch can take seconds.
- **Extra copies:** each batch is copied once into shared memory and again into pinned memory.
- **Head-of-line blocking:** one worker builds a whole batch, and batches must be delivered in order, so one slow sample stalls batches that are already finished.

The Challenge

Removing the GIL does not make threads fast by default. NVIDIA reported that a thread-based loader decoding with Pillow was slightly slower than the process-based one, and the cause has not been explained. Our challenge is to find out what limits thread scaling and to design a runtime that removes it.

Workload characteristics:

- **Dependencies:** samples are independent, but a batch depends on all its samples, and in-order delivery creates a dependency between consecutive batches.

- **Irregular cost:** per-sample cost varies widely (image size, random augmentation), so static assignment leads to load imbalance and stalls.
- **Memory access:** decode and augmentation have good locality inside a sample, but collation and pinned-memory copies are bandwidth-heavy streaming writes.
- **Shared state:** all threads touch the same dataset and transform objects, so reference counts, per-object locks, and the allocator become contended cache lines.

System constraints:

- Python-level scheduling overhead must stay small compared to per-sample work (milliseconds).
- Free-threaded Python makes data races in Python code real, so shutdown, exception propagation, and ordered delivery must be correct under true parallelism.
- The runtime must keep `DataLoader` semantics (sampler order, `worker_init_fn`, `get_worker_info()`, persistent workers) so existing training code works by changing one argument.
- C extensions (Pillow, torchvision) may contain internal locks or shared state we cannot see from Python; diagnosing them requires hardware-level profiling.

We hope to learn where the overheads of process-based loading really come from, how much of them threads can remove, and what new contention threads introduce.

Resources

- **Machines:** GHC cluster machines (Intel i7-9700, 8 cores, 15 GB RAM) for development and most experiments; NCSA Delta `gpuA40x4` nodes (AMD EPYC 7763, 64 cores, 4x NVIDIA A40) for scaling beyond 8 cores and end-to-end training (if time allows).
- **Software:** CPython 3.14 free-threaded build, PyTorch 2.15, torchvision 0.29, Pillow 13.0, all verified on GHC with the GIL staying disabled after import; `perf c2c` for cache-line contention profiling.
- **Starting code:** we modify PyTorch's `torch/utils/data/` (pure Python, no PyTorch rebuild). We start from the existing `_MultiProcessingDataLoaderIter` and write a new thread-based iterator alongside it.
- **Baselines:** the multiprocess `DataLoader`, torchdata.nodes (thread mode), and SPDL.
- **Datasets:** Imagenette (ImageNet subset, 1.5 GB, already on GHC local disk), a text tokenization dataset, and synthetic datasets for controlled experiments.

Goals and Deliverables

Plan to achieve:

1. A thread backend for `DataLoader` (`worker_type="thread"`) that preserves existing semantics and produces bit-identical batches in ordered mode.
2. Work-stealing scheduling at sample granularity, so several threads cooperate on one batch and slow samples no longer block finished batches.
3. Zero-copy collation: preallocate each batch tensor (optionally in pinned memory) and have threads write samples directly into their slots.
4. Thread-private transform copies and buffer pools to remove contention found by profiling.

5. Memory-budget backpressure that bounds in-flight bytes instead of batch count.
6. A full evaluation against the baselines, an ablation study of each mechanism, and at least one upstream fix or data-backed issue for a bottleneck we find in Pillow, torchvision, torchdata, or PyTorch.

Success criteria (7 workers on GHC):

- Image decode workload: throughput at least 0.95x of multiprocess loading, total PSS memory at most 0.7x, CPU time per sample at most 0.95x.
- Variable-cost workload: p99 batch inter-arrival time at most 0.7x.
- Metadata-heavy dataset: PSS memory at most 0.3x.
- Text workload: higher throughput than torchdata.nodes in thread mode.

We expect the largest gains in memory and tail latency, because processes replicate memory and block on slow batches. We expect only small throughput gains on image decoding, which is CPU-bound and where process overhead is a small share of per-sample time. SPDL reports a 33% throughput gain from free-threaded Python, which suggests these targets are reachable.

Hope to achieve:

- End-to-end ResNet-50 training on NCSA Delta A40 GPUs, measuring GPU idle time with each loader.
- Scaling experiments to 32+ workers on the 64-core Delta nodes.
- A C++ implementation of the scheduler core if Python-level scheduling overhead turns out to be significant.

If work goes slowly: the thread backend with correctness checks, the complete baseline comparison across workloads, and a diagnosis of why thread-based Pillow decoding scales poorly.

Analysis questions: Which overheads of process-based loading dominate for each workload? How much of the gain comes from removing the GIL versus our scheduling and memory design? What limits thread scaling: reference-count contention, allocator contention, locks in C extensions, or memory bandwidth?

Poster session: speedup, memory, and tail-latency graphs, plus the ablation table. An optional live demo would run both loaders side by side on a GHC machine.

Platform Choice

Data loading runs on the host CPU in real training jobs, so multi-core CPUs are the natural platform. GHC machines are always available, have all dependencies verified, and support `perf c2c` for user-space contention profiling. Their 8 cores match the common setting of a handful of loader workers per GPU. NCSA Delta A40 nodes add 64 cores to test scaling and real GPUs to measure how loader behavior affects training throughput. We use Python because `DataLoader` and most user datasets are Python; we drop to C or C++ only where profiling shows it is needed.

Schedule

Week	Plan
11/9 - 11/15	Proposal; environment setup; measurement harness (throughput, CPU time, PSS, inter-arrival latency); multiprocess and torchdata.nodes baselines; identify which resource bounds each workload
11/16 - 11/22	Thread backend with semantic and bit-exact correctness checks; per-stage timing and `perf c2c` diagnosis; **milestone report due 11/23**
11/23 - 11/29	Work-stealing scheduler and ordered delivery; memory-budget backpressure (lighter week for Thanksgiving)
11/30 - 12/6	Zero-copy collation; thread-private transforms and buffer pools; full evaluation and ablations; upstream fix
12/7 - 12/11	Stretch goals on NCSA Delta; final report (12/10) and poster session (12/11)

References

1. S. Gross, “PEP 703 – Making the Global Interpreter Lock Optional in CPython,” Python Enhancement Proposals, 2023. https://peps.python.org/pep-0703/
2. T. Wouters, M. Page, and S. Gross, “PEP 779 – Criteria for supported status for free-threaded Python,” Python Enhancement Proposals, 2025. https://peps.python.org/pep-0779/
3. PyTorch Contributors, “torch.utils.data,” PyTorch documentation. https://docs.pytorch.org/docs/stable/data.html ; source code: https://github.com/pytorch/pytorch/tree/main/torch/utils/data
4. PyTorch Contributors, “What is torchdata.nodes (beta)?,” torchdata documentation. https://pytorch.org/data/main/what_is_torchdata_nodes.html
5. M. Hira, C. Puhrsch, V. Andrei, R. Malinovskyy, G. Le Lan, A. Krishnan, J. Cummings, V. Bourgin, O. Gerasimova, M. Martin, G. Gunasekaran, Y. Inoue, A. J. Turner, and R. Krishnamoorthi, “Scalable and Performant Data Loading,” arXiv:2504.20067, 2025. https://arxiv.org/abs/2504.20067
6. M. Szołucha and R. Tabet, “Improved Data Loading with Threads,” NVIDIA Technical Blog, Sep. 13, 2024. https://developer.nvidia.com/blog/improved-data-loading-with-threads